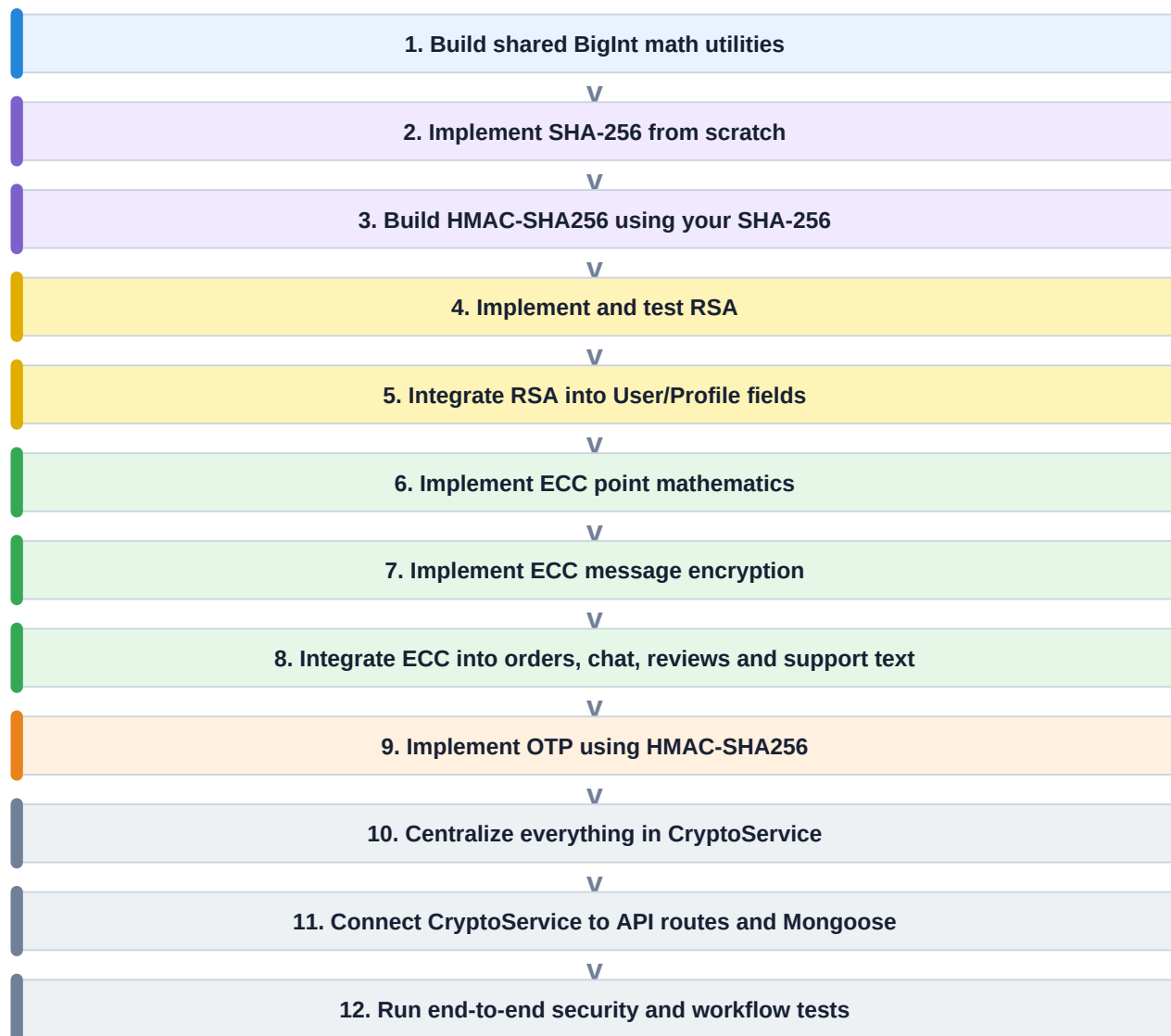


Implementation Pipeline

RSA for profile data | ECC for orders and reviews | SHA-256/HMAC for integrity | OTP for verification

BLUE - Foundation / shared math	YELLOW - RSA / profile protection	GREEN - ECC / order + review protection
PURPLE - SHA-256 / HMAC	ORANGE - OTP verification	GRAY - Integration / testing

Master Pipeline



Important design rule: Do not encrypt entire MongoDB documents. Keep operational fields such as role, order status, prices, timestamps and rider references searchable. Encrypt only sensitive values.

Part A - Build the Cryptographic Foundation

1

Shared math layer - src/lib/crypto/math.ts

Build this first because RSA and ECC both depend on modular arithmetic.

```
gcd()
extendedGCD()
mod()
modInverse()
modPow()
randomBigInt()
randomBigIntBetween()
byte/BigInt conversion helpers
```

2

SHA-256 from scratch - sha256.ts

Input bytes -> padding -> 512-bit blocks -> message schedule -> 64 compression rounds -> final 256-bit digest.

Test with known vectors before using it anywhere else. Expose sha256Bytes() and sha256Hex().

3

HMAC-SHA256 - hmac.ts

Build HMAC on top of your own SHA-256. It supports deterministic lookup tokens, integrity checks and OTP.

$$\text{HMAC}(K, m) = H((K' \text{ XOR opad}) \parallel H((K' \text{ XOR ipad}) \parallel m))$$

4

RSA - rsa.ts

Key generation: choose primes p and $q \rightarrow n \rightarrow \phi(n) \rightarrow$ choose $e \rightarrow d = e^{-1} \bmod \phi(n)$.

Core operations: generateRSAKeyPair(), rsaEncrypt(), rsaDecrypt().

Do not use textbook RSA directly on profile strings. Add a defined encoding/padding layer before modular exponentiation.

Recommended folder structure

```
src/lib/crypto/
math.ts
encoding.ts
random.ts
sha256.ts
hmac.ts
rsa.ts
ecc.ts
otp.ts
cryptoService.ts
```

Part B - RSA for User / Profile Data

RSA protects: contact number, restaurant name/address, rider profile details, and the real email value if you choose to encrypt it.

Keep searchable/plaintext: role, account status, rider activeStatus, verification flags and other operational fields needed by RBAC.

Recommended User schema pattern

```
{
  emailEncrypted: String,
  emailLookupHmac: String,
  contactNumberEncrypted: String,
  restaurantNameEncrypted: String,
  restaurantAddressEncrypted: String,
  vehicleTypeEncrypted: String,
  role: String,
  status: String,
  passwordHash: String,
  cryptoVersion: Number
}
```

Registration pipeline

```
User input
|
+--> normalize email --> HMAC-SHA256 --> emailLookupHmac
|
+--> sensitive profile fields --> RSA encrypt
|
+--> password --> existing password-hash flow
|
+--> save encrypted fields + lookup HMAC + operational fields
```

Login pipeline

```
Email entered
-> normalize
-> HMAC-SHA256
-> findOne({ emailLookupHmac })
-> verify password
-> create normal session/JWT
-> decrypt profile fields only when required
```

Why HMAC is needed: randomized RSA ciphertext is not suitable for direct database lookup. Search by a deterministic HMAC value while keeping the real email encrypted.

Part C - ECC for Orders, Chat and Reviews

6

ECC mathematics - ecc.ts

Use $y^2 = x^3 + ax + b \pmod{p}$. Implement `isPointOnCurve()`, `pointAdd()`, `pointDouble()`, `pointNegate()`, `scalarMultiply()`, and private/public key generation.

7

ECC message layer

message bytes -> split into blocks -> encode block as curve point M -> choose ephemeral k -> $C1 = kG$ -> $C2 = M + kQ$ -> store $(C1, C2)$.

Decryption concept: $M = C2 - dC1$, then decode the point back into message bytes.

Order-field integration

status, user/rider references, item/product references, quantities, prices, payment/delivery method, timestamps, isPaid, numeric rating	shippingAddress, deliveryInstructions, sensitive customer notes, chat text, review text, support-ticket message

Order create flow

```
POST /api/orders
-> authenticate customer
-> validate input
-> split operational vs sensitive fields
-> ECC encrypt sensitive fields
-> save plaintext operational fields + ECC ciphertext
```

Order read flow

```
GET /api/orders/[id]
-> authenticate
-> authorize access FIRST
-> fetch MongoDB document
-> ECC decrypt only fields this role may see
-> return response
```

Critical rule: authorization must happen before decryption. A rider should only receive decrypted details for an order assigned to that rider.

Part D - Chat, Reviews, OTP and CryptoService

Customer-Rider Chat

message text -> ECC encrypt -> store encryptedText -> display

Reviews

numeric rating stays plaintext; review text -> ECC encrypt -> MongoDB

9

OTP using your HMAC

secret + time/counter -> HMAC-SHA256 -> dynamic truncation -> mod 1,000,000 -> six-digit OTP.

Use for account verification, password reset, or sensitive profile changes. Track expiry, attempt count, and used/not-used state.

Phase 10 - Central CryptoService

```
CryptoService
encryptProfile() -> RSA
decryptProfile() -> RSA
encryptOrderField() -> ECC
decryptOrderField() -> ECC
encryptReview() -> ECC
decryptReview() -> ECC
hash() -> SHA-256
hmac() -> HMAC-SHA256
generateOTP() -> OTP
```

Goal: API routes call CryptoService. API routes should not contain low-level RSA or ECC mathematics.

Suggested data ownership

Data	Protection	Purpose
Profile PII	RSA	Confidentiality for user/profile data
Order sensitive text	ECC	Address, instructions, chat and notes
Review text	ECC	Review confidentiality
Email lookup token	HMAC	Search without storing email in plaintext
OTP	OTP + HMAC	Verification

Part E - API Integration Pipeline

Where cryptography sits in BiteRush

```
FRONTEND
|
v
NEXT.JS API ROUTE
|
v
AUTHENTICATION
|
v
ROLE-BASED AUTHORIZATION
|
v
INPUT VALIDATION
|
v
CRYPTO SERVICE
|-----|
| | |
v v v
RSA ECC SHA/HMAC/OTP
profiles orders/reviews integrity
| | |
'-----+-----'
|
v
MONGOOSE
|
v
MONGODB
```

Existing project routes to modify

Route area	Crypto action
Registration / profile APIs	RSA encrypt before save; HMAC email for lookup; RSA decrypt on authorized reads
/api/orders	ECC encrypt sensitive fields during order creation
/api/orders/[id]	Authorize first, then decrypt only permitted fields
/api/orders/[id]/chat	ECC encrypt on POST; authorized ECC decrypt on GET
/api/orders/[id]/rate	Keep numeric rating plaintext; ECC encrypt review text
Restaurant order routes	Keep status transitions plaintext; decrypt sensitive fields only when needed
Rider order routes	Verify assigned rider first; then decrypt address/instructions/chat

Key handling

Store with ciphertext:
algorithm identifier
keyVersion
encoding version

Never store private keys inside normal MongoDB user/order documents
Keep key material server-side and separate from business data.

Implementation Notes Before Testing

From-scratch cryptography rule

Implementing RSA, ECC, SHA-256, HMAC and OTP yourself is suitable for an academic demonstration. For randomness, use an operating-system cryptographically secure random source if your instructor permits it. A homemade random-number generator can make otherwise correct cryptography insecure.

Key-management rule

Keep RSA/ECC private keys server-side and separate from normal User and Order documents. Store only algorithm identifiers, keyVersion and encodingVersion beside ciphertext. This allows future key rotation without changing the business workflow.

Authorization rule

Authenticate and authorize before decrypting. Customers may read their own protected order fields; restaurants receive only what their role needs; riders receive decrypted delivery details only for orders assigned to them.

Do not encrypt operational fields

Keep role, account status, order status, prices, timestamps, product/rider references, isPaid and numeric rating searchable. Encrypt sensitive profile values, shipping information, delivery instructions, chat text, review text and support messages.

Failure-handling rule

If ciphertext is malformed, a key version is unknown, or integrity checks fail, return a controlled error. Do not expose private keys, raw intermediate values, stack traces or decrypted secrets in API responses or logs.

Part F - Development and Testing Checklist

Milestone 1 - Foundation

- ☐ Implement math.ts
- ☐ Test modInverse and modPow
- ☐ Implement SHA-256
- ☐ Verify known SHA-256 vectors
- ☐ Implement HMAC-SHA256
- ☐ Verify known HMAC vectors

Milestone 2 - RSA profiles

- ☐ RSA key generation works
- ☐ Encrypt -> decrypt returns original
- ☐ Protect one User field first
- ☐ Add emailLookupHmac
- ☐ Expand to remaining profile fields
- ☐ Verify login still works

Milestone 3 - ECC orders

- ☐ Point addition/doubling tests
- ☐ Scalar multiplication tests
- ☐ Key generation tests
- ☐ Message-point encoding test
- ☐ Encrypt one delivery instruction field
- ☐ Expand to address/chat/reviews

Milestone 4 - OTP and integration

- ☐ Generate valid six-digit OTP
- ☐ Expiry and attempt limits
- ☐ Create CryptoService
- ☐ Authorization before decrypt
- ☐ Add cryptoVersion/keyVersion
- ☐ Handle corrupted ciphertext safely

End-to-end test sequence

1. Register customer
2. Confirm sensitive profile values are ciphertext in MongoDB
3. Log in using HMAC email lookup
4. Place order with delivery address + instructions
5. Confirm sensitive order fields are ECC ciphertext
6. Restaurant accepts and changes status normally
7. Assign rider
8. Rider can decrypt only assigned delivery details
9. Customer and rider exchange encrypted chat messages
10. Rider marks delivered
11. Customer submits rating + ECC-encrypted review text
12. Verify unauthorized roles cannot decrypt protected fields

Final architecture: RSA = profiles | ECC = orders/chat/review text | SHA-256 + HMAC = hashing, integrity and lookup | OTP = verification.

Main integration rule: Frontend -> API -> authentication/RBAC -> validation -> CryptoService -> Mongoose -> MongoDB.